DSML Module Test: ML 2 : Unsupervised and RecSys (No Class Today) Performance Report

satyakovvuri999@gmail.com

DSML Module Test: ML 2 : Unsupervised and RecSys (No Class Today)

100.0
/100

SCORE

Duration : 2 hours

Total Questions : 8

Attempted : 8 Solved : 8

Start Time : Sunday, 27 Sep 2026 10:05 AM IST

QUESTION STATS

S1 **Default Section (210/210)**

#
Topic ⓘ
Type/Language ⓘ

Score ⓘ
Status ⓘ
Actions
No:
1
Problem:
Miscellaneous
Type/Language:

Score:
30.0 / 30.0
Status:
Solved
View Details

Cross Validate Time Series

Which of these cross-validation techniques is **not** suitable for time series data?

- (●) **K-fold cross validation**
- ( ) **Blocked cross-validation**
- ( ) **Predict Second half**
- ( ) **Day Forward Chaining**

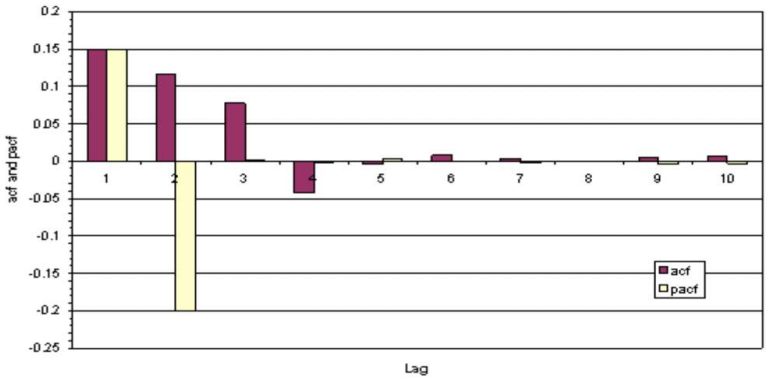
No:
2
Problem:
Time Series
Type/Language:

Score:
60.0 / 60.0
Status:
Solved

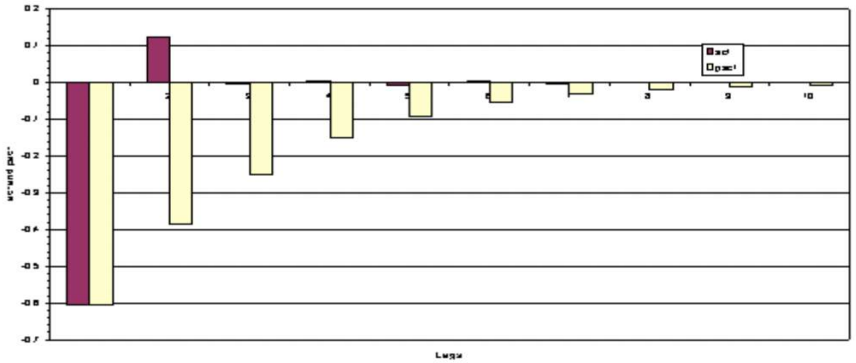
View Details

What do the graphs imply?

a.



b.



For the above two data which of the following options are true?

Note: In the options, AR and MA refer to Auto regressive and moving average processes respectively.

◉ **For a, AR(2) and MA(2) for b can be suggested.**

○ **For a, MA(2) and AR(2) for b can be suggested.**

○ **For a, MA(4) and AR(4) for b can be suggested.**

○ **For a, MA(2) and AR(3) for b can be suggested.**

No:
3
Problem:
Recommender System
Type/Language:

Score:
30.0 / 30.0
Status:
Solved

View Details

Find the leverage

Based on the 5 transaction entries given, answer the question below.

Order 1: [Bread, Milk, Butter]
Order 2: [Butter, Eggs, Bread]
Order 3: [Eggs, Milk, Cola, Butter]
Order 4: [Bread]
Order 5: [Diapers, Beer, Cola, Butter]

What would be the **leverage** value for **{Bread -> Butter}**?

- (●) **-0.08**
- ( ) **-0.6**
- ( ) **0.4**
- ( ) **0.68**
- ( ) **0.8**

No:
4
Problem:
Recommender System
Type/Language:

Score:
30.0 / 30.0
Status:
Solved

View Details

Truncated SVD

Which of the following is/are **true** in regards to Truncated SVD?

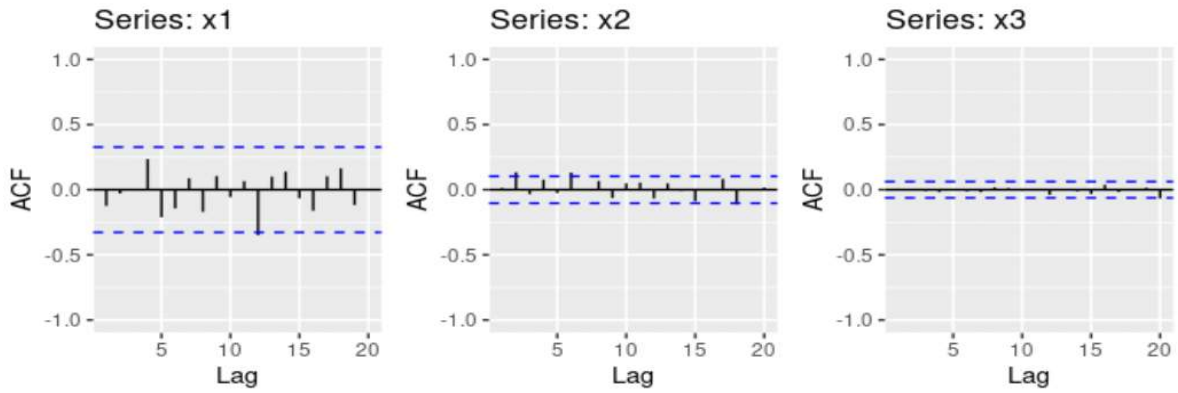
- [ ] **It is computationally more expensive than regular SVD**
- [ ] **HyperParameter k must be less than 0 (k < 0)**
- [x] **k < n, where n is the number of singular values**
- [x] **Truncated SVD gives an approximation of the original matrix**

No:
5
Problem:
Time Series
Type/Language:

Score:
60.0 / 60.0
Status:
Solved
View Details

ACF plots

The plots shown below, show the ACFs for 36 numbers, 360 numbers, and 1,000 numbers.



Mark the option which is correct regarding the plots.

- All the three plots show non-random data.
- **All the plots show random data.**
- The ACF bars should be higher for a larger population random data.
- The ACF bars in the first plot (left) shows significant autocorrelation above critical value.

S2 Section 2 (260/260)

#
Topic ⓘ
Type/Language ⓘ
Score ⓘ
Status ⓘ
Actions
No:

1

Problem:

Recommender System

Type/Language:

</>

Python 3 (python-3.8)

Score:

100.0 / 100.0

Status:

Solved

View Details

3 Most similar movies

Build a recommender function **get_recommendations()**. It takes in a **title**, a **cosine similarity matrix**, and an **indices series** as arguments and outputs a list of 3 titles most similar to the original title (excluding the title itself). **indices** has already been provided to you.

You have been given a dataset **metadata** that consists of the movie **titles** and respective **genre**.

Instructions:-

- Get index of the movie that matches the title by using the **title** key of **indices**.
- Extract the 3 most similar movies from **sim_scores** and store it back in **sim_scores.**

Input format:

```
List of tuples containing movie titles and genres

Movie title
```

Output format:

```
3 movies similar to the inputted movie title in newline
```

Sample input:

```
[("Dhoom3", "Action"), ("Rambo", "Action"), ("Hangover", "Comedy"), ("Krish", "Hero"), ("Ted", "Comedy"), ("2 States", "Romance"), ("127 Hours", "Drama"), ("Love Story", "Romance"), ("ZNMD", "Comedy"), ("Anek", "Drama")]

Dhoom3
```

Sample output:

```
Rambo

Hangover

Krish
```

User Code

```
 import pandas as pd

# Generate mapping between titles and index
indices = pd.Series(metadata.index, index=metadata['title']).drop_duplicates()
# Notice that the index for this Series is the titles of #movies

def get_recommendations(title, cosine_sim, indices):
    # Get index of movie that matches title
    idx = indices[title]

    # Sort the movies based on the similarity scores
    sim_scores = list(enumerate(cosine_sim[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

    # Get the scores for 3 most similar movies
    sim_scores = sim_scores[1:4]

    # Get the movie indices
    movie_indices = [i[0] for i in sim_scores]

    # Return the top 3 most similar movies
    return metadata['title'].iloc[movie_indices]
```

Previous Submissions:

#	Input Type	Time of submission	Result	Time	Memory	Score	Source	Submission Mode
1	Correctness	Sun, 27 Sep 2026 10:22:50 +0530	**Correct Answer**	3537 ms	254444 KB	**50.0**	View Code	**Live Mode**

No:
2
Problem:
Time Series
Type/Language:
</>
Python 3 (python-3.8)
Score:
100.0 / 100.0
Status:
Solved
View Details

Time Series Outliers

Problem Description:

```
In time series forecasting, it is important to handle the points that you treat differently from
"Normal Looking" data points;
i.e. Outliers. Define a function that takes input series and does the following:

      -> Calculate absolute value of difference of each observation from the mean
      -> Create a boolean mask for observations for which the difference from mean is more than tw
o times the Standard Deviation of the data.
      -> Using the mask, replace the outliers with the median of the entire series
```

Note: The time series is assumed to be stationary here.

Input Format:

An list is taken as input and typecasted as NumPy array.

Output Format:

The resultant array of series with the outlier being replaced by the median value of the entire series has to be returned.

Sample Input:

```
[ 330, 11500, 16500, 914, 796, 353, 6470, 219, 389, 805, 256, 919, 398, 900, 496, -6240, 948, 711, 306, 777, 719, 545, 248]
```

Sample Output:

```
[ 330 711 711 914 796 353 6470 219 389 805 256 919 398 900 496 -6240 948 711 306 777 719 545 248]
```

User Code

```
 import numpy as np

def replace_outliers(arraySeries):
   """arraySeries is a numpy array,
      return the required numpy array"""

   # Calculate the absolute difference of each timepoint from the series mean
   mean_val = np.mean(arraySeries)
   abs_diff = np.abs(arraySeries - mean_val)

   # Calculate a mask for the differences that are > 2 standard deviations from the mean
   std_val = np.std(arraySeries)
   mask = abs_diff > (2 * std_val)

   # Replace these values with the median accross the data
   median_val = np.median(arraySeries)
   arraySeries[mask] = median_val

   return arraySeries
```

Previous Submissions:

#	Input Type	Time of submission	Result	Time	Memory	Score	Source	Submission Mode
1	Correctness	Sun, 27 Sep 2026 10:20:41 +0530	**Correct Answer**	619 ms	88156 KB	**25.0**	View Code	**Live Mode**

No:
3
Problem:
Time Series
Type/Language:
</>
Python 3 (python-3.8)
Score:
60.0 / 60.0
Status:
Solved

View Details

Joyrides

Given a data frame '**rides**', with the features '**Start station**', '**End station**' and '**Duration**'. Suppose you have a theory that some people take long bike rides before putting their bikes back in the same dock. Let's call these rides "joyrides". You only have data on one bike, so while you can't draw any bigger conclusions, it's certainly worth a look.

Print the median duration of all the rides and the rides in which the start and end station are same (i.e. joyrides), rounded off to two decimal places.

Input Format:

```
Three lists are taken as input which are then combined to form rides data frame.
```

Output Format:

```
Two strings.
```

Sample Input:

Start station = ['Broad St', '66 St-Lincoln Center', '86th St', 'Times Sq - 42 St', '86th St', 'Bowery', '66 St-Lincoln Center', '66 St-Lincoln Center', '14 St / 8 Av', 'Times Sq - 42 St']
End station = ['14 St / 8 Av', 'Bowery', 'Bowery', '66 St-Lincoln Center', '66 St-Lincoln Center', 'Broad St', 'Broad St', 'Broad St', '14 St / 8 Av', 'Times Sq - 42 St']
Duration = [0.12, 1.94, 2.47, 1.53, 2.76, 1.93, 0.33, 0.94, 2.71, 1.39]

Sample Output:

```
The median duration overall was 1.73 hrs
The median duration for joyrides was 2.05 hrs
```

User Code

```
 import pandas as pd
#rides is the loaded dataframe consisting of columns 'Start station', 'End station' and 'Duration'

# find joyrides
joyrides = (rides['Start station'] == rides['End station'])

# Median of all rides
print("The median duration overall was {:.2f} hrs".format(rides['Duration'].median()))

# Median of joyrides
print("The median duration for joyrides was {:.2f} hrs".format(rides[joyrides]['Duration'].median()))
```

Previous Submissions:

#	Input Type	Time of submission	Result	Time	Memory	Score	Source	Submission Mode

1	Correctness	Sun, 27 Sep 2026 10:24:57 +0530	**Correct Answer**	1428 ms	128824 KB	**50.0**	View Code	**Live Mode**